

Amortized Analysis

Amortized analysis studies the average cost per operation over a sequence of operations, even when some individual operations are expensive.

Natural Explanation

Some data structures are usually fast but occasionally do a lot of work. A classic example is a dynamic array: - Most PUSH operations just write one element (cheap). - Rarely, the array is full and must resize by copying many elements (expensive).

Amortized analysis says: do not judge by one worst operation alone. Spread rare expensive costs across many cheap operations.

Academic View

For a sequence of m operations with total cost $C(m)$, the amortized cost per operation is:

$$\hat{c} = \frac{C(m)}{m}.$$

If $C(m) = O(m)$, then amortized cost is $O(1)$ per operation.

Important distinction: - Worst-case per operation: maximum cost of a single operation. - Amortized cost: average over a worst-case sequence of operations.

Amortized guarantees are deterministic and stronger than probabilistic average-case assumptions about input distributions.

Standard Example: Dynamic Array Doubling

Suppose capacity doubles whenever full.

- Non-resize push: cost $O(1)$.
- Resize push: copy current elements + insert new one, cost $O(n)$ at that moment.

Across n pushes, total copied elements are:

$$1 + 2 + 4 + \dots + 2^{\lfloor \log_2 n \rfloor} = O(n).$$

So total cost of n pushes is $O(n)$, hence amortized PUSH is:

$$O(n)/n = O(1).$$

Common Methods

- Aggregate method:
 - Bound total cost of a sequence, then divide by number of operations.
- Accounting method:
 - Charge operations an artificial cost and store credits to pay for future expensive steps.
- Potential method:
 - Define a potential function Φ and analyze amortized cost via changes in potential.

For this course level, aggregate intuition is often sufficient for dynamic arrays.

Why It Matters

- Explains why globally rebuilding structures can still be efficient.
 - Justifies practical performance of dynamic arrays, hash tables, and similar structures.
 - Avoids overestimating cost based on rare outlier operations.
-

Pitfalls

- Confusing amortized with average-case under random inputs.
 - Claiming every operation is cheap because amortized cost is low.
 - Ignoring resizing policy details (for example, poor grow/shrink thresholds can hurt performance).
-

Related

- [[Big-O, Big-Omega, Big-Theta]]
- [[Growth Rate Hierarchy]]
- [[Recurrence Relations]]
- [[Lectures/Week 4]]